

Reviewer Pack — Castelnuovo-Mumford Regularity of Partial Derivative Ideals ...

Entry 13aab9b22051 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 04:17:01 UTC

Castelnuovo-Mumford Regularity of Partial Derivative Ideals Bounds ABP Width

Entry ID: 13aab9b22051 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 04:17:01 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry **Field B** (complexity object): Algebraic Branching Programs (ABPs)

Statement:

For a polynomial f represented by an ABP of width w , the Castelnuovo-Mumford regularity of the ideal I generated by f 's partial derivatives satisfies $\text{reg}(I) \leq w * \log n + O(1)$.

Rationale (proposer's reasoning):

The regularity of the partial derivative ideal captures the minimal number of generators needed to describe the polynomial's singularities, while ABP width reflects computational complexity. Linking them could reveal structural constraints on polynomial representations.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 720a87873a39ba13

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
import sys

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_polynomial(n, w):
        # Generate a random polynomial over GF(2)
        poly = [random.choice([0, 1]) for _ in range(n)]
        return poly

    def partial_derivatives(poly_str, vars, x_values):
        # Compute partial derivatives
        partials = []
        for var in vars:
            partial = ""
            for i, coeff in enumerate(poly_str.split()):
                if var in coeff:
                    power = int(coeff[coeff.index(var) + 1:])
                    if power > 0:
                        new_coeff = str((power * int(coeff[:coeff.index(var)])) % 2)
                        new_var = f"{var}^{power-1}"
                        partial += f"{new_coeff}*{new_var} "
            partials.append(partial.strip())
        return partials

    def construct_ideal(partial):
        # Construct the ideal generated by partial derivatives
        I = set()
        for part in partial:
            I.update(set(part.split()))
        return I

    def castelnuovo_mumford_regularity(I, n):
        # Compute Castelnuovo-Mumford regularity (simplified)
        return len(I) * math.log(n, 2)

    n = random.randint(5, 40)
```

```

w = random.randint(1, 10)
poly_str = generate_polynomial(n, w)
vars = [f"x{i}" for i in range(w)]
x_values = [random.randint(0, 1) for _ in range(n)]

partials = partial_derivatives(poly_str, vars, x_values)
I = construct_ideal(partials)
reg_I = castelnuovo_mumford_regularity(I, n)

metric_name = "castelnuovo_mumford_regularity"
metric_value = reg_I
instances_tested = 1
conjecture_holds = reg_I <= w * math.log(n, 2) + 10
counterexample = "" if conjecture_holds else f"reg(I)={reg_I} > {w * math.log(n, 2) + 10}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) # Default to 1
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, ...{result}...}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")
    else:
        print("RESULT: INCONCLUSIVE insufficient support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f290040.py", line 81, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f290040.py", line 58, in run_trial
    partials = partial_derivatives(poly_str, vars, x_values)
               ~~~~~~
```

```
AttributeError: 'list' object has no attribute 'split'
```

Critic verdict: CHALLENGE

skipped: test crashed before producing data

Verdict: INCONCLUSIVE

Test crashed due to `AttributeError`, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Fix the test code's handling of polynomial strings to avoid `AttributeError` and re-run experiments

Total LLM calls: 9

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 255051 ms total latency. Provider mix: {'ollama remote': 9}

12. Reproducibility

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/13aab9b22051.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/13aab9b22051.tar.gz (if generated)